
Module 8 — Cybersecurity Orchestration

Automating an authorized penetration test: Chains → RAG → Agents + MCP

Part 1

Module Overview & Concept Recap

What we build in this module

- An orchestrated, AUTHORIZED penetration-testing workflow: recon → triage → CVE lookup → report.
- Built three ways, each better than the last:
 - Stage 1 — a fixed Chain (LLM answers from memory)
 - Stage 2 — RAG (ground the analysis in a real knowledge base)
 - Stage 3 — an Agent that orchestrates MCP tools and decides what to call

Recap — the building blocks (RAG, LangChain, LangGraph, MCP)

- RAG : retrieve trusted facts so the model stops guessing — grounding, citations.
- LangChain : compose prompts + tools into a repeatable pipeline.
- LangGraph : stateful, tool-using agents — branch, call tools, keep state.
- MCP : a standard plug so any client/agent can discover and call your tools.
- This module is the payoff: we orchestrate all four on a real security task.

Scope & legality come first

- Only scan or test systems you are explicitly authorized to touch.
- In this module: simulated hosts, the Nmap Project's scanme.nmap.org, and your own isolated VM.
- Guardrails we enforce in code: scope allowlists, human-in-the-loop approval, logging.
- Dual-use reality: the defender's tools are the attacker's tools — behave accordingly.

Part 2

A Simple Orchestration Pipeline (Simulated)

The notebook: `part2_orchestration_simulated.ipynb`

- Safe and offline: a SIMULATED nmap over fictional lab hosts — reproducible, no packets sent.
- Same three stages you'll see again on real data next part.
- Backed by `recon_server_simulated.py` (an MCP server) and `cve_knowledge_base.md`.

Stage 1 — a Chain, and its limitation

- Pipeline: recon → parse services → analyze → report, as a fixed chain.
- The 'analyst' answers from the model's memory only.
- Limitation: it sounds authoritative but hallucinates CVEs and versions — not trustworthy.

Stage 2 — RAG grounds the analysis

- Retrieve real entries from `cve_knowledge_base.md` for the detected services.
- Feed retrieved facts to the analyst → grounded, cited, far fewer hallucinations.
- Compare Stage 1 vs Stage 2 side by side in the notebook.

Stage 3 — an Agent orchestrates MCP tools

- recon_server_simulated.py exposes tools: port_scan(), lookup_cve(), list_targets().
- An agent DECIDES which tool to call next instead of following a fixed script.
- Offline deterministic loop shown first; live LangGraph+MCP agent when a key is set.



What the LLM is good and bad at here

- Good: summarizing scans, drafting reports, mapping findings to remediation language.
- Bad: inventing CVE numbers, version facts, and exploit details from memory.
- Design rule: let tools + RAG supply facts; let the LLM do language, not lookup.

Hands-on (Part 2)

- Open `code/part2_orchestration_simulated.ipynb` and run top to bottom.
- Watch the Stage 1 → 2 → 3 quality improve on the same target.
- Optional: set `OPENAI_API_KEY` to run the live agent over the MCP server.

Part 3

One Level Further: Real Targets

The notebook: `part3_web_pentest_real.ipynb`

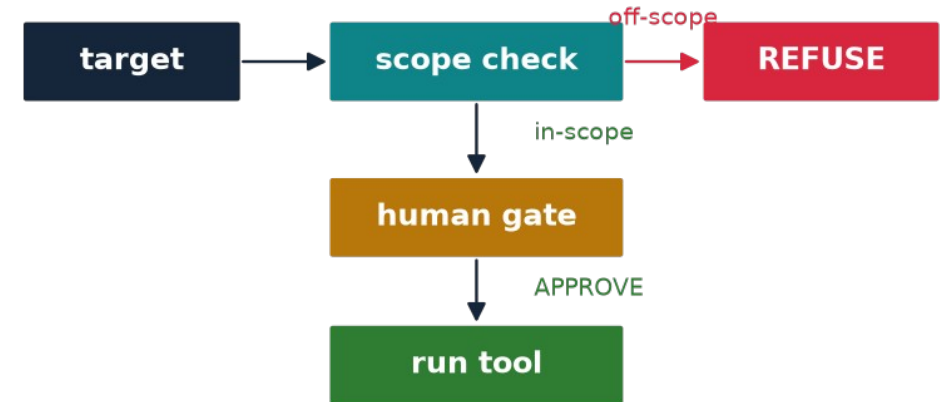
- Same three stages — now on REAL, authorized targets (default: `scanme.nmap.org`).
- Recon is non-intrusive: one HTTP GET per call, with an offline cached fallback.
- Backed by `recon_server_web.py` and `web_security_kb.md` (OWASP guidance).

Same stages, real data — what changes

- Stage 1 chain now describes a real site — and still hallucinates without grounding.
- Stage 2 RAG grounds findings in OWASP categories and concrete remediation.
- Stage 3 agent chooses tools (headers → guidance) to assemble a grounded report.

Allowlisting is a control, not a suggestion

- ALLOWED_HOSTS is enforced in the server before any request is made.
- Off-scope target → the tool returns a refusal, not a scan.
- This is how you make automation safe to hand to an agent.



Logging & auditability

- Every tool call and its result should be logged for review.
- Auditability lets a human verify what the agent did — and reproduce it.
- Keep server stderr/logs; never silently swallow a failed or refused call.

Hands-on (Part 3)

- Run `code/part3_web_pentest_real.ipynb` against an allowed host.
- Try an off-scope host and observe the allowlist refusal.
- Inspect the grounded report, the agent generated report, and compare them to Stage 1's ungrounded draft.

recon_server_web.py — the Stage-3 tools

- `fetch_headers()` — retrieve HTTP response headers.
- `check_security_headers()` — flag missing HSTS, CSP, X-Frame-Options, etc.
- `get_robots()` — read robots.txt.
- `lookup_guidance()` — OWASP remediation from `web_security_kb.md`; resource `kb://websec`.
- Authorization-guarded: calls are refused for hosts outside the allowlist.

Part 4

Final Project: Orchestrated Pen-Test

The task

- Orchestrate a pen-test of your OWN isolated Metasploitable 2 VM (host-only network).
- Start from the simulated pipeline; run it on your REAL scan data.
- Expand the CVE RAG, add tools beyond nmap, and maximize vulnerability coverage.

The scaffold: assignments/project/

- Final_Project_PenTest.ipynb — the student notebook with graded TODOs.
- Final_Project_Requirements.md — full instructions and the rubric.
- project_cve_kb.md — your editable knowledge base to expand.
- solutions/ — instructor Sample Solution notebook + Sample Report (what an A looks like).

Deliverables & rubric highlights

- Completed, runnable notebook with your real scan output.
- Expanded RAG (before/after) — accurate CVEs for Metasploitable services.
- ≥ 2 real tools integrated (MCP-wrapped for full credit) + a second knowledge base.
- Comprehensive report: methodology + an evidence-based chains-vs-RAG-vs-agent comparison.
- Bonus: a live LangGraph + MCP agent making ≥ 2 autonomous tool calls.

Reuse the pattern you already have

- Simulated (Part 2) → Web/real (Part 3) → your own orchestration (project).
- Model new tools on recon_server_simulated.py's @mcp.tool() functions.
- Ground every finding — no ungrounded CVE claims in the report.



Ethics & scope — the rules again

- Everything stays in your isolated lab; report only on your authorized VM.
- Never scan, test, or collect data from systems outside the approved scope.
- Keep API keys and secrets out of any shared or public repository.

Recap & what's next

- You orchestrated RAG + agents + MCP on a real security task, with guardrails.
- Next module (9): Vibe Coding — build, break, and harden an AI-generated web app.
- Questions? Post in the Module 8 discussion forum.